Technical report: a knowledge graph of Guelph Transit bus timetables

Mohammad Khashab, Knowledge Representation (MSc), Lebanese University. October 2026. Repository: https://github.com/Mohammadkh19/kr-transit-project

1. Problem framing and scope

Bus timetables are published as GTFS: a zip of linked tables (routes, trips, stops, stop times, calendar). The tables answer simple lookups, but not questions that follow the links, such as "which stops are served by more than one route?", and nobody can ask them in English. We built a knowledge graph of one agency's timetable that answers such questions, checks its own data, and refuses what it cannot know.

The system answers five competency questions (ontology/competency_questions.md):

#	Question
CQ1	Which stops does route 99 serve, in order?
CQ2	Which stops are served by more than one route?
CQ3	Which routes run on Sunday?
CQ4	Which trips are wheelchair accessible, and on which routes?
CQ5	When does the first trip of each route leave on a Saturday?

They are measurable: each is in the access layer's test set with its correct query, and the score is recorded (Section 7).

Left out, and why:

- Real-time data (delays, vehicle positions): it is a separate feed (GTFS Realtime) that changes every few seconds. The system refuses these questions.
- Fares: the Guelph feed has no fare files.
- Stop wheelchair access: empty or "no information" for every stop in our sample, so we answer accessibility for trips instead.
- The full network: we keep 3 routes and the morning trips (Section 2), so every layer can be checked by hand.

2. Data understanding and constraint discovery

Source. Guelph Transit's GTFS Schedule feed (City of Guelph, Ontario), service from 4 January to 2 May 2026, taken from the MobilityDatabase copy (mdb-3140). Licence: Open Government Licence - City of Guelph v2.0. The full feed has 1 agency, 35 routes, 628 stops and 5,201 trips (data/gtfs-validator-report.html).

Our sample. data/make_sample.py keeps routes 1, 2 and 99 and only trips that start between 07:00 and 08:59, then follows the IDs from routes to trips, stop times and stops, so the sample stays consistent. Routes 1 and 2 are the same loop in opposite directions and 99 is the main line, so many stops are shared: a real graph, not

three separate lines. Result (data/example.sql): 3 routes, 95 trips, 104 stops, 2,356 stop times, 3 services (weekdays, Saturday, Sunday). The script only trims spaces; it fixes no data, so the defects stay for the shapes to find.

What one row means.

- routes: one bus line (route 99, "Mainline").
- trips: one journey of a bus on a route, on a service (a set of days).
- stop_times: one trip at one stop, at a time, in an order (stop_sequence).
- stops: one bus stop, with a name and a position.
- calendar: one service: which days of the week it runs, and the date range.

Constraint inventory. From the GTFS specification, the official validator's report on the full feed (MobilityData validator 8.0.0), and our own look at the data. The full list with severities is in shapes/triage.md; the main findings:

Constraint	Evidence in the data
Along a trip, distance traveled goes up	96 errors in the full feed (validator)
Every stop is used by a trip	5 unused stops in the full feed (validator)
A service runs on at least one day	service 7 in the full feed runs on no day
Times are written HH:MM:SS	Guelph writes "7:01:06", not "07:01:06"
The first stop of a trip has no distance	true for all 95 trips of our sample

Constraints we found false in the data, reported rather than dropped.

- HH:MM:SS: false for every time in our sample (one-digit hours). We kept the data and report it as a Warning (2,356 warnings, Section 4).
- Every stop has wheelchair information: false, the column is empty or 0.
- The full-feed defects (distance, unused stops, service 7) are not in our sample, because we kept 3 routes and morning trips. Our shapes would catch them; we showed it with a deliberately broken row (Section 4).

3. Ontology

Reused vocabulary. Linked GTFS (http://vocab.gtfs.org/terms#, CC BY 3.0, Open Transport contributors), the published RDF version of GTFS. It fits because it has one class or property for each GTFS table and column we use: gtfs:Route, gtfs:Trip, gtfs:StopTime, gtfs:Stop, gtfs:Service, gtfs:CalendarRule, gtfs:route, gtfs:stop, gtfs:sunday and so on. For stop positions we reuse the W3C Basic Geo vocabulary (geo:lat, geo:long).

Reuse without import. Our data uses the Linked GTFS IRIs, but ontology/ontology.ttl does not owl:import the published file. We declare only the terms we use, with the domains and ranges that are true for our data. The published file would give wrong inferences: gtfs:route has two ranges (gtfs:Route and gtfs:FareRule), so every route of a trip would become a fare rule; gtfs:shortName has two domains, so every route would become a trip; gtfs:wheelchairAccessible has the domains gtfs:Stop and gtfs:Station, so an accessible trip would become a stop. HermiT also cannot load the published file, because gtfs:arrivalTime uses xsd:duration,

which is not in the OWL 2 datatype map. This is the same lesson as the Session 3 lab: a term chosen for its name brings a formal meaning with it. The reasons are written down in ontology/competency_questions.md.

Alignment outward. gtfs:Stop skos:closeMatch schema:BusStop; our two classes are rdfs:subClassOf Linked GTFS classes.

Competency questions and what we added. The five questions of Section 1. Linked GTFS answers most of them. We added only two classes, each with tr:answersCQ pointing to its question:

- tr:SundayService (CQ3): a service whose calendar rule has sunday = true.
- tr:AccessibleTrip (CQ4): a trip whose wheelchairAccessible value is gtfs:WheelchairAccessible. Both are defined with owl:equivalentClass, so the reasoner classifies data into them. CQ2 ("served by more than one route") needs counting, which our profile does not allow, so it is answered by a SPARQL query, not a class.

OWL profile. OWL 2 EL: our definitions need only intersection, someValuesFrom and hasValue. EL keeps reasoning fast on the full feed.

Reasoner run. HermiT on ontology.ttl plus reasoner-check.ttl, a few real things from our sample written without types (ontology/reasoner-run.txt): no impossible classes. Trip 465522 had no type; the reasoner inferred it is a Trip (domain of gtfs:route) and then an AccessibleTrip. Service 3 was inferred to be a SundayService, service 1 was not. Route 2841 became a Route and nothing else: no FareRule, because we did not import that axiom.

Version IRI. https://ul.edu.lb/kr/transit/1.0.1. Version 1.0.0 was bumped when we added geo:lat and geo:long for the mapping.

Limitation. Fuseki does not run a reasoner, so the two classes are not in the endpoint: a query for tr:AccessibleTrip finds nothing. Queries use the property instead (gtfs:wheelchairAccessible).

4. Constraint validation with SHACL

The shapes. shapes/shapes.ttl has 11 constraints (C1 to C11), one or more per item of the inventory in Section 2. Each has an sh:message a person can act on, starting with its number. Severity says what the pipeline does: a Violation stops the load, a Warning is reported and the data goes through.

#	Constraint	Severity
C1, C2	A trip has exactly one route and one service	Violation
C3-C5	A stop time has one trip, one stop, one position	Violation
C6	Times look like H:MM:SS	Violation
C7	Times use two digits, HH:MM:SS	Warning
C8	Distance traveled goes up along a trip (SPARQL constraint)	Violation
C9	A stop has one name and a valid latitude and longitude	Violation
C10	Every stop is used by a trip	Warning
C11	A service runs on at least one day (SPARQL constraint)	Violation

Why SHACL and not OWL. These rules say what must be present and correct. OWL works under the open world assumption: if a trip has no route, OWL assumes the route exists somewhere and reports nothing. SHACL checks only what is in the graph (closed world), so it can say "error: route missing". The reverse case: "a trip with wheelchair access is an accessible trip" adds knowledge, so it is an OWL axiom (tr:AccessibleTrip), not a shape. The reason for each constraint is in shapes/triage.md.

Triage of every failure on our real data (shapes/triage.md). Our sample gives 0 Violation and 2,356 Warning.

- C7, 2,356 warnings: a data defect in the source (one-digit hours). Harmless for our questions, so we keep the data and report it.
- C8, C10, C11: real defects of the full feed (96 distance errors, 5 unused stops, service 7). Not in our sample, because we kept 3 routes and morning trips.
- Stop wheelchair access: no rule possible, the column is empty.

The gate. pipeline/run.py validates the graph together with the ontology before loading Fuseki. To test it, we set the latitude of stop 103 to 200 in data/example.sql. Locally: 1 Violation (C9), the pipeline stopped with code 1 and Fuseki was not loaded (report/red-run.png). In CI: the same row on the branch broken-row made the SHACL gate workflow fail, so it could not be merged (report/red-run-ci.png). The branch was never merged; the value was set back.

Limitation. The SPARQL constraint C8 takes about 15 seconds on our sample; on the full feed (95,000 stop times) it would be much slower. The 2,356 warnings are noise in which a real warning could hide.

5. Operational data integration

The mapping. mapping/mapping.ttl is an R2RML mapping (W3C Recommendation, 2012) from the GTFS tables to our ontology; pipeline/run.py runs it with Morph-KGC. One triples map per table, and one IRI template per kind of thing: https://ul.edu.lb/kr/id//guelph/, for example .../id/route/guelph/2866 for route 99. A trip and its route are linked because the trip's route_id builds the same IRI. Our sample gives 17,319 triples. Mapping decisions:

- Days: GTFS writes 1 or 0; an SQL view in the mapping writes true or false (xsd:boolean), as the ontology says.
- Wheelchair access: 1 gives gtfs:WheelchairAccessible, 2 gives gtfs:NotWheelchairAccessible, an empty cell gives no triple. Empty means "no information", not "no".
- Times stay text: GTFS allows hours past 24 (25:10:00 is 1:10 the next morning), which xsd:time does not.
- route_type 3 becomes the Linked GTFS value gtfs:Bus.

Materialized or virtualized. The deployed system materializes: the pipeline writes all the triples once and loads them into Fuseki. We did not run the virtualized alternative (Ontop, Session 5 lab). The argument:

- Freshness: a GTFS timetable changes a few times a year, with each new season. Re-running the pipeline for each new feed is enough; virtualization is useful when the source changes all the time.
- Latency: our questions follow long paths (stop time, trip, service, calendar rule). On stored triples Fuseki answers them directly; a virtual system must turn each into SQL joins at every question.
- Workload: the sample is small (17,319 triples), and the SHACL gate needs the whole graph in one place before loading, which only materialization gives. Virtualization would become the right choice for GTFS Realtime, which changes every few seconds.

Entity resolution across two sources. Our GTFS stops against OpenStreetMap bus stops around Guelph (451 stops, downloaded 2026-10-19, ODbL; mapping/er/osm-bus-stops.json). mapping/er/match_stops.py compares position and name only. Blocking: only pairs less than 100 m apart are compared (255 of the 46,904 possible pairs). Score: half distance, half name similarity.

The labelled sample: in OSM, many stops carry ref, the stop number printed on the sign, which is the GTFS stop code. 82 of our 104 stops have exactly one OSM stop with their code: those are the true matches. The matcher never reads ref; we use it only to grade (mapping/er/result.txt).

Threshold	Matches	Right	Precision	Recall
0.5	74	70	0.946	0.854
0.8	71	70	0.986	0.854

We use 0.8. Errors: 3 of the 4 wrong matches at 0.5 are the stop on the other side of the road (northbound against southbound), because the true OSM stop has no name; the 4th is a GO Transit stop at the same corner. The 12 missed stops have no name in OSM. Limitation: the labels come from OSM's ref, not from a hand check, so an error in ref would be an error in our grading.

6. Prediction and evaluation honesty

The graph for learning. model/build_graph.py reads build/graph.nt and builds a PyTorch Geometric graph: one node per stop (104), one edge from a stop to the next stop of a trip (109). A stop's features: how many trips stop there, how many routes, whether a trip starts or ends there; its position is used for the split and the link baseline.

Node classification: does this stop have a shelter? The label comes from OpenStreetMap through our entity resolution (match score 0.8 or more), so it is not in the graph and cannot leak through the features: 58 stops are labelled, 40 with a shelter. Models: always answer the majority label; a logistic regression on the stop's row (table); GraphSAGE, two rounds of message passing over the next-stop links.

The split does not leak: stops less than 150 m apart (the two sides of one corner) form a group, and a whole group goes to train or to test (33 groups). A random split would put one side of a corner in training and the other in test, and a model could copy the answer from across the road. Five seeds (model/results/shelter.txt):

Model	ROC-AUC	Accuracy
majority	0.500 ± 0.000	0.706 ± 0.037
table	0.387 ± 0.208	0.589 ± 0.130
GraphSAGE	0.473 ± 0.065	0.584 ± 0.061

No model beats always answering "yes". With 58 labels the test sets hold about 20 stops, and the spread is large. Whether a stop has a shelter depends on things our data does not hold (space on the pavement, the city's budget, how many people wait there), not on how many buses stop.

Link prediction: which stop comes next? 15% of the stop pairs are hidden, in both directions, so the model cannot get A to B from a training link B to A. For each hidden link, every stop is ranked; stops already linked in

training do not count against the model (filtered). Baseline: the nearest stop. Model: GraphSAGE embeddings scored by dot product, trained with negative sampling. Three seeds (model/results/next-stop.txt):

Model	MRR	Hits@1
nearest stop	0.159 ± 0.025	0.000 ± 0.000
GraphSAGE	0.184 ± 0.075	0.104 ± 0.078

GraphSAGE is higher on average, but the difference is

7. Access layer

What it does. A question in English becomes a SPARQL query over our graph (access-layer/access_layer.py, served by serve.py on 127.0.0.1:8000). The language model (Gemini, gemini-3.5-flash-lite) only writes the query. The answer always comes from Fuseki, so it comes from our data, not from the model's memory.

Grounding. The model never sees the data. It gets:

- build/void.ttl, a VoID summary of the graph made by the pipeline: which classes it has, how many of each, and which properties link them;
- access-layer/examples.ttl, 8 example questions with their right queries; the 3 closest to the question go into the prompt;
- access-layer/prompt_rules.txt: how things are named in our data (a route by its number with gtfs:shortName, a stop by its name with rdfs:label, a trip number as an IRI) and when to refuse.

Check and repair. Before a query runs, it is checked: it must be valid SPARQL and use only classes and properties that the graph has (from the VoID). If it fails, the problems go back to the model, which tries again, at most 3 tries; then the system refuses.

Refusal. When the question needs a kind of fact the graph does not have, the model answers REFUSE with the reason, and the system returns a query that returns nothing, with the reason in a comment. Example: "How much does a bus ticket cost?" gives "# REFUSE: fares", because the Guelph feed has no fares.

The test set and its score. access-layer/questions.yaml: 12 questions the graph can answer, each with its right query (gold), and 4 it cannot (real-time delays, ticket price, shelters, drivers). None is an example question. Each answer is scored by F1 on the set of values it returns against the gold query's. Model gemini-3.5-flash-lite, 2026-10-19, setting "repair":

	Mean F1 (12 questions)	Refused right	Refused wrongly
Before (report/access-layer-score-before.txt)	0.806	4 of 4	2
After (report/access-layer-score-after.txt)	0.889	4 of 4	1

The improvement. Before, question 6 ("How many wheelchair accessible trips does route 99 have?") was refused. The model wrote a correct query, but our check treated every capitalized gtfs: term as a class, and gtfs:WheelchairAccessible is a fixed value (an individual), not a class. The check rejected the query and the repair loop pushed the model into refusing. We let the check accept the four Linked GTFS fixed values we use

(FIXED_VALUES in access_layer.py). The fix is general: it was a bug in the check that would block any accessibility question. We did not change any test question or gold query.

What still fails. Question 5 ("Which stops are served by more than one route?", our CQ2) is refused by the model itself, before any query: it seems to think the graph cannot count routes per stop. Question 10 ("Which stops does route 2 serve?") scores F1 0.67. The model is not fully repeatable, so a new run can differ a little.

Reproducible without the network. The runs used LLM_MODE=record, which saves every model answer in access-layer/llm-cache.json; with LLM_MODE=replay the same score comes back without calling Gemini.

8. Deployment and reproducibility

The stack (docker-compose.yml), one command: docker compose up --build.

- fuseki: Apache Jena Fuseki 5.5.0, the SPARQL endpoint, dataset kr.
- pipeline: runs once. 1 load data/example.sql into SQLite, 2 map with Morph-KGC, 3 SHACL gate, 4 write the VoID, 5 load the graph and the ontology into Fuseki. A Violation at step 3 stops it with code 1.
- access-layer: starts only if the pipeline finished without error (service_completed_successfully), and restarts by itself if it stops (restart: unless-stopped), for example when Gemini cannot be reached.

Security. Every port listens on 127.0.0.1 only, not on the network. The Gemini key is only in .env, which .gitignore keeps out of the repository.

Provenance. Every source is named where it is used, with its licence (table at the end): the Guelph feed and its validator report in data/, the OpenStreetMap download in mapping/er/, and the versions of every model run in model/results/ and report/.

Ontology version. https://ul.edu.lb/kr/transit/1.0.1, loaded with the data at step 5.

What a clean checkout taught us.

- The first build failed with a pip timeout on a slow connection (duckdb could not be downloaded). We gave pip a longer timeout and retries in pipeline/Dockerfile.
- The reasoner script failed on Windows: owlready2 could not open a file given as a file:// address. It now opens the file and passes its content.
- [CLEAN CHECKOUT RESULT: fill in after the test on Tuesday]

The Session 7 checklist.

- One command from a clean checkout: yes (see above).
- Services on 127.0.0.1 only, keys only in .env: yes.
- SHACL gate before the endpoint and in CI: yes (Section 4).
- The test set's score recorded with the model and the date: yes (Section 7).
- A real unanswerable question refused, with a reason: yes, "How much does a bus ticket cost?" gives "REFUSE: fares".

9. Open problem: time and space

The question. How should a knowledge graph say *when* a fact is true and *where* things are, so that both can be asked in SPARQL? (Session 7, open problem 4.)

Where it appears in our system.

- *When a fact is true.* Our timetable is valid from 4 January to 2 May 2026. The graph holds the dates in calendar.txt only as text we did not map, and every triple looks true forever: someone asking in October gets the January-May answer, with no warning.
- *Times of day.* GTFS times can pass 24:00 ("25:10:00" is 1:10 the next morning, still part of today's service). xsd:time cannot hold that, so we kept times as text (Section 5). Text sorts wrongly after 9:59 ("10:00" comes before "7:00"), so comparing times works in our sample only because every trip starts before 9:00.
- *Where things are.* Stops carry geo:lat and geo:long as plain numbers. To find stops less than 100 m apart (entity resolution, Section 5) and less than 150 m apart (the split, Section 6) we computed distances in Python, outside the graph, because plain SPARQL has no distance function.

What the literature offers.

- Gutierrez, Hurtado and Vaisman (2007) add time to RDF itself: each triple gets a time label, the period when it holds, and they define what queries over such graphs mean. With it, our 95 trips could carry their service period, and a question in October could be told the timetable no longer applies.
- OGC GeoSPARQL 1.1 (2024) is a standard for places in RDF, with functions inside SPARQL such as geof:distance. With it, "which stops are within 100 m of this one?" would be one query instead of our Python code.
- W3C Time Ontology in OWL gives standard terms for instants, intervals and days of the week. Our service period is an interval, and gtfs:monday ... gtfs:sunday could be its days of the week. But it has no notion of a "service day" that runs past midnight, so 25:10:00 would still need our own modelling.

What we did, and what we could not do. We recorded the limitation instead of hiding it: the validity period is in the report and in data/README.md, and the shapes check the time format (C6, C7). We did not add time labels to triples, because neither Fuseki nor our SHACL gate or access layer would understand them without new code. We did not switch on GeoSPARQL in Fuseki, so distances stay in Python.

Our position. For transit data, time is the harder and more open half. Space has a standard (GeoSPARQL) that works today; for us the gap is engineering, enabling it in our endpoint. Time has no single accepted way to say "this triple holds from January to May" that SPARQL, SHACL and a reasoner all understand, and nothing handles the GTFS service day past midnight cleanly. Our next step would be to give every service its validity period with the Time Ontology, and to let the access layer refuse questions about dates outside it.

Sources.

1. C. Gutierrez, C. A. Hurtado, A. Vaisman. "Introducing Time into RDF". IEEE Transactions on Knowledge and Data Engineering 19(2), 2007.
2. Open Geospatial Consortium. GeoSPARQL 1.1, A Geographic Query Language for RDF Data. OGC Standard, 2024.
3. W3C. Time Ontology in OWL. W3C Recommendation, https://www.w3.org/TR/owl-time/

Licences

Ontology, vocabulary or dataset	Licence	Where used
Guelph Transit GTFS feed (City of Guelph)	Open Government Licence - City of Guelph v2.0	data/example.sql
GTFS validator report (MobilityData, validator 8.0.0)	Apache License 2.0 (the validator)	data/gtfs-validator-report.html
Linked GTFS (Open Transport contributors)	CC BY 3.0	ontology, mapping, shapes, access layer
W3C Basic Geo vocabulary (geo:lat, geo:long)	W3C Document License	ontology, mapping
schema.org (schema:BusStop, alignment only)	CC BY-SA 3.0	ontology
OpenStreetMap bus stops	ODbL 1.0, © OpenStreetMap contributors	mapping/er/, model/
Our ontology (tr:)	CC BY 4.0	ontology/ontology.ttl

Attribution required by the Guelph licence: "Contains information licensed under the Open Government Licence - City of Guelph."